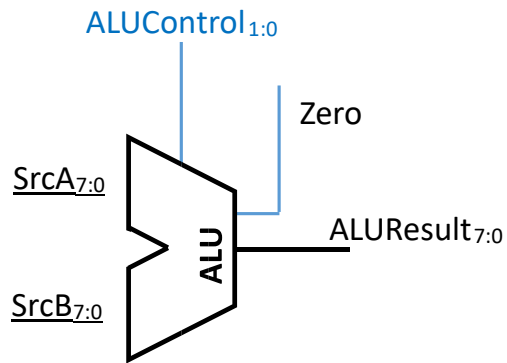


Digital Design lab – Building CPU By SystemVerilog

Design and test activities:

1. *Arithmetic Logic Unit (ALU)*
2. *Register file*
3. *Combined ALU and register file*
4. *Read-only memory (ROM)*
5. *Program counter*
6. *Combining the ROM with program counter*
7. *Control unit*
8. *Complete CPU design*

1. Arithmetic logic unit (ALU)



Module name	alu
Output ports	ALUResult[7:0], Zero
Input ports	SrcA[7:0], SrcB[7:0], ALUControl[1:0]

ALUControl	Operation	Description
00	$a \& b$	Bitwise logical operation: a AND b
01	$a b$	Bitwise logical operation: a OR b
10	$a + b$	Arithmetic operation, addition: $a + b$
11	$a - b$	Arithmetic operation, subtraction: $a - b$

Module Code:

```

module alu (input logic [7:0] SrcA,SrcB,
input logic [1:0] ALUControl,
output logic [7:0] ALUResult,
output logic Zero);  always_comb
begin

    case(ALUControl)
        2'b00: ALUResult = SrcA & SrcB;
        2'b01: ALUResult = SrcA | SrcB;
        2'b10: ALUResult = SrcA + SrcB;
        2'b11: ALUResult = SrcA - SrcB;
        default: ALUResult = 8'bx;    endcase

```

```

        if(ALUResult == 0) begin
Zero = 1;      end    else
begin          Zero = 0;
end    end

```

Test Bench Code:

```

`timescale 1ps/1ps
`include "alu.sv"

module alu_tb; logic [7:0]
SrcA,SrcB,ALUResult; logic [1:0]
ALUControl; logic Zero;

alu dut (SrcA,SrcB,ALUControl,ALUResult,Zero);

initial begin
    $dumpfile("alu_tb.vcd");
    $dumpvars(0,alu_tb);

    SrcA = 5;SrcB = 3;
    #10;
    ALUControl = 2'b00;
    #10;
    ALUControl = 2'b01;
    #10;
    ALUControl = 2'b10;
    #10;
    ALUControl = 2'b11;
    #10;
    $finish; end

initial begin

    $monitor("time = %3d, A = %d, B = %d, Zero = %b, ALUResult
= %b",$time,SrcA,SrcB,Zero,ALUResult); end

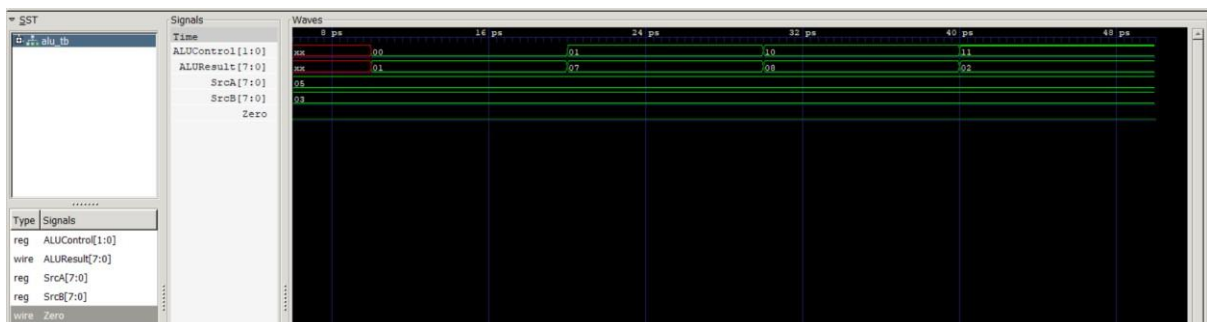
```

endmodule

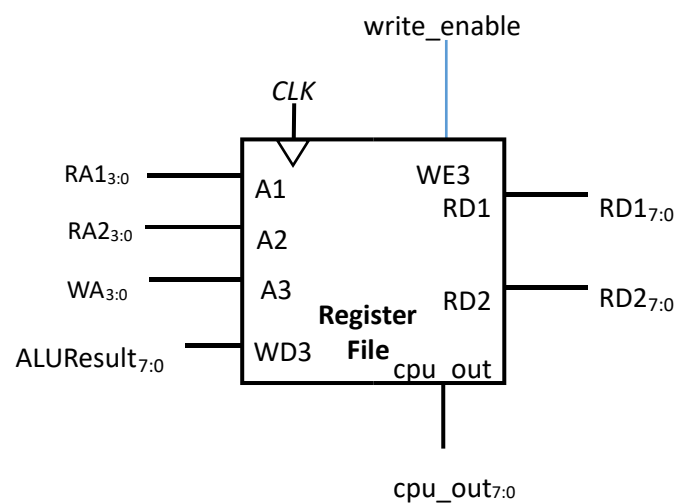
Test Result:

```
PS C:\Users\13452\Documents\Coursework files\ELEC0010> iverilog -g 2012 -o alu_tb.vvp alu_tb.sv
PS C:\Users\13452\Documents\Coursework files\ELEC0010> vvp alu_tb.vvp
VCD info: dumpfile alu_tb.vcd opened for output.
time = 0, A = 5, B = 3, Zero = 0, ALUResult = xxxxxxxx
time = 10, A = 5, B = 3, Zero = 0, ALUResult = 00000001
time = 20, A = 5, B = 3, Zero = 0, ALUResult = 00000111
time = 30, A = 5, B = 3, Zero = 0, ALUResult = 00001000
time = 40, A = 5, B = 3, Zero = 0, ALUResult = 00000010
alu_tb.sv:25: $finish called at 50 (1ps)
```

gtkwave:



2. Register file



Module name	reg_file
Output ports	RD1[7:0, RD2[7:0], cpu_out[7:0]

Input ports	RA1[3:0], RA2[3:0], WA[3:0], ALUResult[7:0], write_enable, CLK
--------------------	--

Module Code:

```

module reg_file (input logic [3:0] RA1,RA2,WA,
input logic [7:0] ALUResult,           input
logic write_enable,CLK,               output logic
[7:0] RD1,RD2,cpu_out);

    logic [7:0] rf [0:15];
assign RD1 = rf[RA1];
assign RD2 = rf[RA2];

    always_comb begin
        rf[0] = 8'b0;
    end

    always_ff @( posedge CLK ) begin
if (write_enable && WA > 0) begin
        rf[WA] <= ALUResult;    end

        cpu_out <= rf[15];
    end

endmodule

```

Test Bench Code:

```

`timescale 1ps/1ps
`include "reg_file.sv"

module reg_file_tb; logic
[3:0] RA1,RA2,WA;
logic [7:0] ALUResult,RD1,RD2,cpu_out; logic
write_enable,CLK;

```

```
reg_file dut (RA1,RA2,WA,ALUResult,write_enable,CLK,RD1,RD2,cpu_out);
```

```
initial begin    CLK =
```

```
0;    forever begin
```

```
#10 CLK = ~CLK;
```

```
end end
```

```
initial begin
```

```
    $dumpfile("reg_file_tb.vcd");
```

```
    $dumpvars(0,reg_file_tb);
```

```
    ALUResult = 5;WA = 0;write_enable = 1;
```

```
for (int i = 0;i <= 15;i = i + 1 ) begin
```

```
    #10 WA = WA + 1;
```

```
    #10 ALUResult = ALUResult + 2;
```

```
RA1 =WA-1;RA2 = WA;    end
```

```
    RA1 = 1; RA2 = 2; WA = 0; ALUResult = 5; write_enable = 0;
```

```
    #15 write_enable = 1;
```

```
    #20 WA = 1; ALUResult = 7;
```

```
    #20 WA = 5; ALUResult = 13;
```

```
    #20 WA = 15; ALUResult = 30;
```

```
    #20 write_enable = 0;
```

```
    #20 WA = 5; ALUResult = 11;
```

```
    #20 RA2 = 5;RA1 = 8;
```

```
    #30 write_enable = 1;
```

```
    #15 WA = 15; ALUResult = 23;
```

```
    #20 WA = 4; ALUResult=24;
```

```
    #20 RA1 = 4; RA2 = 15;
```

```
    $finish; end
```

```
initial begin
```

```
$monitor("time=%3d, write_enable = %b, RD1 = %d, RD2 = %d,cpu_out
= %d",$time,write_enable,RD1,RD2,cpu_out); end
```

```
endmodule
```

Test Result:

```

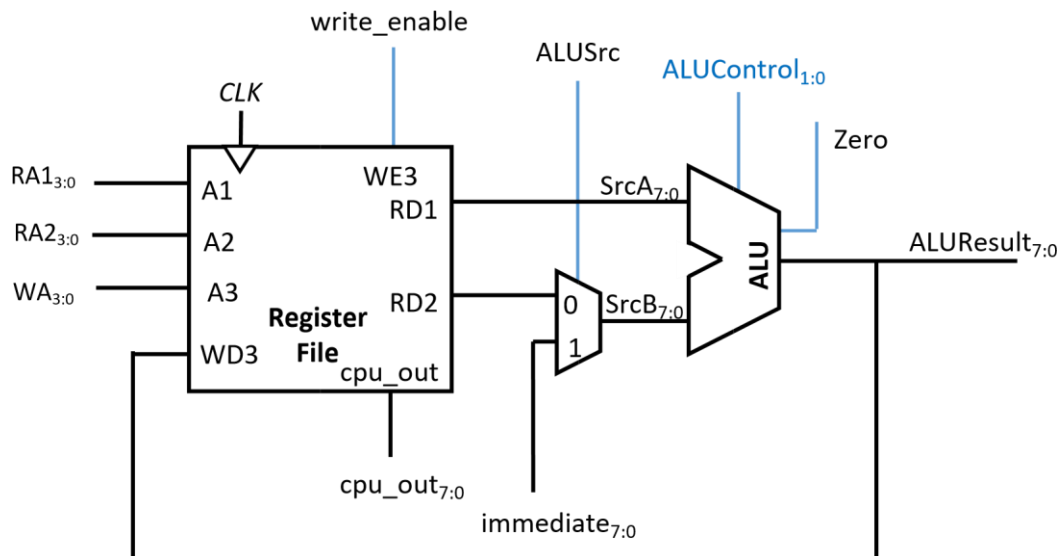
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
PS C:\Users\13452\Documents\Coursework files\ELEC0010> iverilog -g 2012 -o reg_file_tb.vvp reg_file_tb.vv
./reg_file.vv:10: warning: always_comb process has no sensitivities.
PS C:\Users\13452\Documents\Coursework files\ELEC0010> vvp reg_file_tb.vvp
time= 80, write_enable = 1, RD1 = 9, RD2 = 11,cpu_out = x
time=100, write_enable = 1, RD1 = 11, RD2 = 13,cpu_out = x
time=120, write_enable = 1, RD1 = 13, RD2 = 15,cpu_out = x
time=140, write_enable = 1, RD1 = 15, RD2 = 17,cpu_out = x
time=160, write_enable = 1, RD1 = 17, RD2 = 19,cpu_out = x
time=180, write_enable = 1, RD1 = 19, RD2 = 21,cpu_out = x
time=200, write_enable = 1, RD1 = 21, RD2 = 23,cpu_out = x
time=220, write_enable = 1, RD1 = 23, RD2 = 25,cpu_out = x
time=240, write_enable = 1, RD1 = 25, RD2 = 27,cpu_out = x
time=260, write_enable = 1, RD1 = 27, RD2 = 29,cpu_out = x
time=280, write_enable = 1, RD1 = 29, RD2 = 31,cpu_out = x
time=300, write_enable = 1, RD1 = 31, RD2 = 33,cpu_out = x
time=310, write_enable = 1, RD1 = 31, RD2 = 33,cpu_out = 33
time=320, write_enable = 0, RD1 = 5, RD2 = 7,cpu_out = 33
time=335, write_enable = 1, RD1 = 5, RD2 = 7,cpu_out = 33
time=370, write_enable = 1, RD1 = 7, RD2 = 7,cpu_out = 33
time=415, write_enable = 0, RD1 = 7, RD2 = 7,cpu_out = 33
time=430, write_enable = 0, RD1 = 7, RD2 = 7,cpu_out = 30
time=455, write_enable = 0, RD1 = 19, RD2 = 13,cpu_out = 30
time=485, write_enable = 1, RD1 = 19, RD2 = 13,cpu_out = 30
time=490, write_enable = 1, RD1 = 19, RD2 = 11,cpu_out = 30
time=530, write_enable = 1, RD1 = 19, RD2 = 11,cpu_out = 23
reg_file_tb.vv:42: $finish called at 540 (1ps)
time=540, write_enable = 1, RD1 = 24, RD2 = 23,cpu_out = 23
PS C:\Users\13452\Documents\Coursework files\ELEC0010>

```

gtkwave:



3. Combining the ALU and register file



Module name	reg_file_alu
Output ports	ALUResult[7:0], cpu_out[7:0], Zero
Input ports	RA1[3:0], RA2[3:0], WA[3:0], immediate[7:0], write_enable, ALUSrc, ALUControl[1:0], CLK

Module Code:

```

`include "alu.sv"
`include "reg_file.sv"

module reg_file_alu (input logic [3:0] RA1,RA2,WA,
input logic [7:0] immediate,          input logic
[1:0] ALUControl,          input logic
write_enable, ALUSrc, CLK,          output
logic [7:0] ALUResult,cpu_out,          output
logic Zero);

    logic [7:0] SrcA,SrcB,RD1,RD2;
```



```
alu alu_in (SrcA,SrcB,ALUControl,ALUResult,Zero);
```

```
reg_file reg_file_in  
(RA1,RA2,WA,ALUResult,write_enable,CLK,RD1,RD2,cpu_out);
```

```
always_comb begin
```

```
    SrcA = RD1;
```

```
    if (ALUSrc) begin
```

```
        SrcB = immediate;
```

```
    end    else begin
```

```
        SrcB = RD2;    end
```

```
end
```

```
endmodule
```

Test Bench Code:

```
`timescale 1ps/1ps
```

```
`include "reg_file_alu.sv"
```

```
module reg_file_alu_tb; logic
```

```
[3:0] RA1,RA2,WA; logic [7:0]
```

```
immediate; logic [1:0]
```

```
ALUControl; logic write_enable,
```

```
ALUSrc, CLK; logic [7:0]
```

```
ALUResult,cpu_out; logic Zero;
```

```
reg_file_alu dut (RA1,RA2,WA,immediate,ALUControl,write_enable, ALUSrc,  
CLK,ALUResult,cpu_out,Zero);
```

```
initial begin    CLK =
```

```
0;    forever begin
```

```
#10 CLK = ~CLK;
```

```
end end
```

```
initial begin
```

```
    $dumpfile("reg_file_alu_tb.vcd");
```

```
    $dumpvars(0,reg_file_alu_tb);
```

```
    #10 WA = 0;write_enable = 1;immediate = 2;
```

```
    #10 ALUControl = 2'b10;
```

```
    for (int i = 0; i <= 15; i = i + 1) begin
```

```
        #10 ALUSrc = 1;
```

```
        #10 RA1 = i;RA2 = i;
```

```
    #10 WA = WA + 1;    end
```

```
    RA1 = 7; RA2 = 8;
```

```
    #30;
```

```
    ALUControl = 2'b00;
```

```
    #30;
```

```
    ALUControl = 2'b01;
```

```
    #30;
```

```
    ALUControl = 2'b10;
```

```
    #30;
```

```
    ALUControl = 2'b11;
```

```
    $finish;
```

```
end
```

```
initial begin
```

```
    $monitor("Time = %3d, ALUResult = %d, Zero = %b, cpu_out  
= %d",$time,ALUResult,Zero,cpu_out); end
```

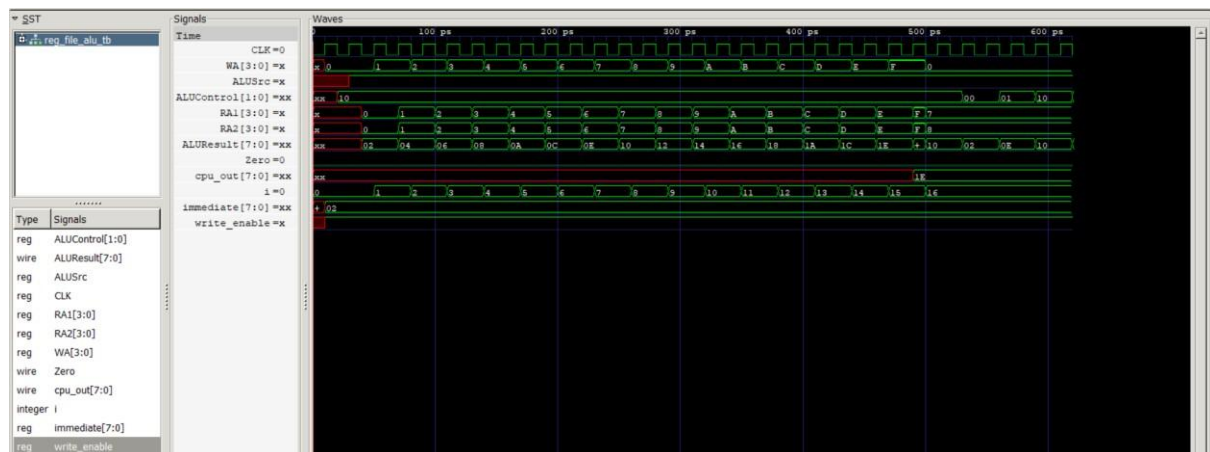
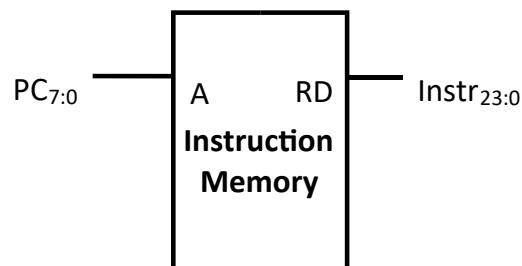
```
endmodule
```

Test Result:

```

PS C:\Users\13452\Documents\Coursework files\ELEC0010> iverilog -g 2012 -o reg_file_alu_tb.vvp reg_file_alu_tb.v
./reg_file_alu_tb.v:10: warning: always_comb process has no sensitivities.
PS C:\Users\13452\Documents\Coursework files\ELEC0010> vvp reg_file_alu_tb.vvp
VCD info: dumpfile reg_file_alu_tb.vcd opened for output.
Time = 0, ALUResult = x, Zero = 0, cpu_out = x
Time = 40, ALUResult = 2, Zero = 0, cpu_out = x
Time = 70, ALUResult = 4, Zero = 0, cpu_out = x
Time = 100, ALUResult = 6, Zero = 0, cpu_out = x
Time = 130, ALUResult = 8, Zero = 0, cpu_out = x
Time = 160, ALUResult = 10, Zero = 0, cpu_out = x
Time = 190, ALUResult = 12, Zero = 0, cpu_out = x
Time = 220, ALUResult = 14, Zero = 0, cpu_out = x
Time = 250, ALUResult = 16, Zero = 0, cpu_out = x
Time = 280, ALUResult = 18, Zero = 0, cpu_out = x
Time = 310, ALUResult = 20, Zero = 0, cpu_out = x
Time = 340, ALUResult = 22, Zero = 0, cpu_out = x
Time = 370, ALUResult = 24, Zero = 0, cpu_out = x
Time = 400, ALUResult = 26, Zero = 0, cpu_out = x
Time = 430, ALUResult = 28, Zero = 0, cpu_out = x
Time = 460, ALUResult = 30, Zero = 0, cpu_out = x
Time = 490, ALUResult = 32, Zero = 0, cpu_out = 30
Time = 500, ALUResult = 16, Zero = 0, cpu_out = 30
Time = 530, ALUResult = 2, Zero = 0, cpu_out = 30
Time = 560, ALUResult = 14, Zero = 0, cpu_out = 30
Time = 590, ALUResult = 16, Zero = 0, cpu_out = 30
reg_file_alu_tb.v:44: $finish called at 620 (1ps)
Time = 620, ALUResult = 12, Zero = 0, cpu_out = 30
PS C:\Users\13452\Documents\Coursework files\ELEC0010>

```

gtkwave:**4. Instruction memory**

Module name	instruction_memory
Output port	Instr[23:0]
Input port	PC[7:0]

Instruction memory address	Machine code program	Assembly program	Execution
00	610001	addi x1, x0, 1	# Write the value 1 into register x1

01	620020	addi x2, x0, 32	# Write the value 32 into x2
02	6F0000	addi x15, x0, 0	# Write the value 0 into x15
03	701207	beq x1, x2, 7	# Branch to instruction 7 if x1 == x2 == 32
04	211100	add x1, x1, x1	# x1 = 2 * x1
05	6FF001	addi x15, x15, 1	# Increment value in register x15
06	700003	beq x0, x0, 3	# Unconditional branch to instruction 3
07	700007	beq x0, x0, 7	# Unconditional branch to instruction 7

Module Code:

```

module instruction_memory (input logic [7:0] PC,
                           output logic [23:0] Instr);

    logic [24:0] data_ROM [0:255];

    initial begin
        $readmemh("program.txt", data_ROM);
    end

    assign Instr = data_ROM[PC];

endmodule

```

Test Bench Code:

```

`timescale 1ps/1ps
`include "instruction_memory.sv"

module instruction_memory_tb;
    logic [7:0] PC; logic [23:0]
    Instr;

    instruction_memory dut (PC,Instr);

```

initial begin

```
$dumpfile("instruction_memory_tb.vcd");
```

```
$dumpvars(0,instruction_memory_tb);
```

```
PC = 8'h00;
```

```
#10 PC = 8'h01;
```

```
#10 PC = 8'h02;
```

```
#10 PC = 8'h03;
```

```
#10 PC = 8'h04;
```

```
#10 PC = 8'h05;
```

```
#10 PC = 8'h06;
```

```
#10 PC = 8'h07;
```

\$finish; end

initial begin

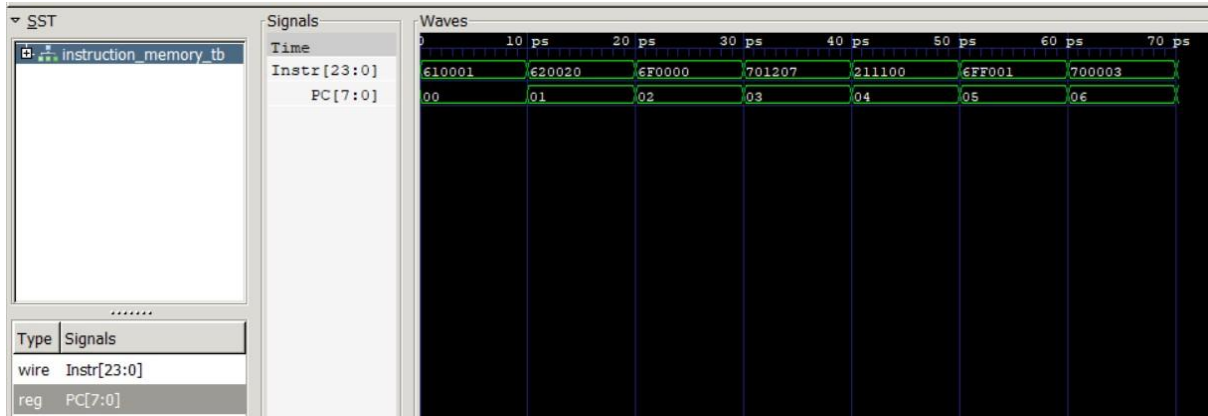
```
$monitor("time = %3d, PC = %d, Instr = %h", $time, PC, Instr); end
```

endmodule

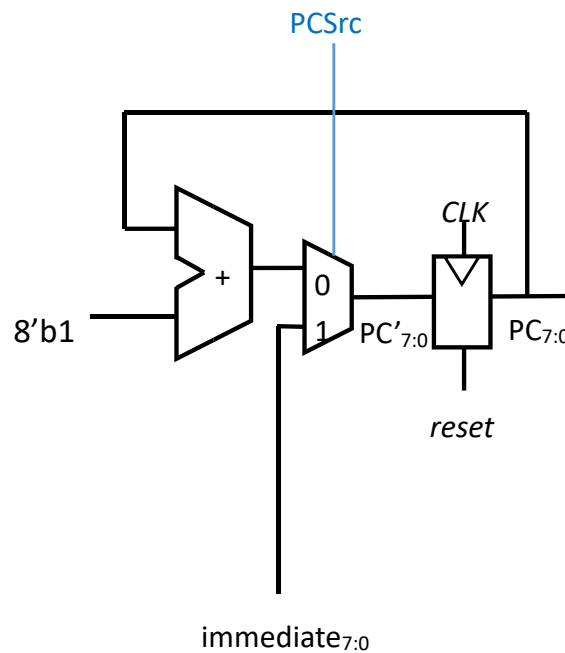
Test Result:

```
PS C:\Users\13452\Documents\Coursework files\ELEC0010> iverilog -g 2012 -o instruction_memory_tb.vvp instruction_memory_tb.sv
PS C:\Users\13452\Documents\Coursework files\ELEC0010> vvp instruction_memory_tb.vvp
WARNING: ./instruction_memory.sv:7: $readmemh(program.txt): Not enough words in the file for the requested range [0:255].
VCD info: dumpfile instruction_memory_tb.vcd opened for output.
time = 0, PC = 0, Instr = 610001
time = 10, PC = 1, Instr = 620020
time = 20, PC = 2, Instr = 6f0000
time = 30, PC = 3, Instr = 701207
time = 40, PC = 4, Instr = 211100
time = 50, PC = 5, Instr = 6ff001
time = 60, PC = 6, Instr = 700003
instruction_memory_tb.sv:23: $finish called at 70 (1ps)
time = 70, PC = 7, Instr = 700007
PS C:\Users\13452\Documents\Coursework files\ELEC0010> |
```

gtkwave:



5. Program Counter



Module name	pc
Output ports	PC[7:0]
Input ports	immediate[7:0], PCSrc, CLK, reset

Module Code:

```

module pc (input logic [7:0] immediate,
input logic PCSrc, CLK, reset,      output
  logic [7:0] PC);

  logic [7:0] PC_temp;

  always_comb begin

```

```

        if (PCSrc) begin
PC_temp    =    immediate;
end

        else begin
            PC_temp = PC + 8'b01;
        end
    end

    always_ff @( posedge CLK ) begin

        if (reset) begin
PC <= 0;        end
        else begin        PC
<= PC_temp;        end
    end

endmodule

```

Test Bench Code:

```

`timescale 1ps/1ps
`include "pc.sv"

module pc_tb; logic
[7:0] immediate; logic
PCSrc, CLK, reset; logic
[7:0] PC;

pc dut (immediate, PCSrc ,CLK ,reset ,PC);

initial begin    CLK =
0;    forever begin

```

```
#10 CLK = ~CLK;
```

```
end
```

```
end
```

```
initial begin
```

```
    $dumpfile("pc_tb.vcd");
```

```
    $dumpvars(0,pc_tb);
```

```
    reset = 0;immediate = 8'h00;
```

```
    PCSrc = 1;
```

```
    #10
```

```
    #10 PCSrc = 0;
```

```
    #10
```

```
    #10
```

```
    #10
```

```
    #10
```

```
    #10
```

```
    #10
```

```
    #10
```

```
    #10 PCSrc = 1;immediate = 8'h05;
```

```
    #10 PCSrc = 0;
```

```
    #10
```

```
    #10
```

```
    #10
```

```
    #10
```

```
    #10
```

```
    $finish;
```

```
end
```

```
initial begin
```

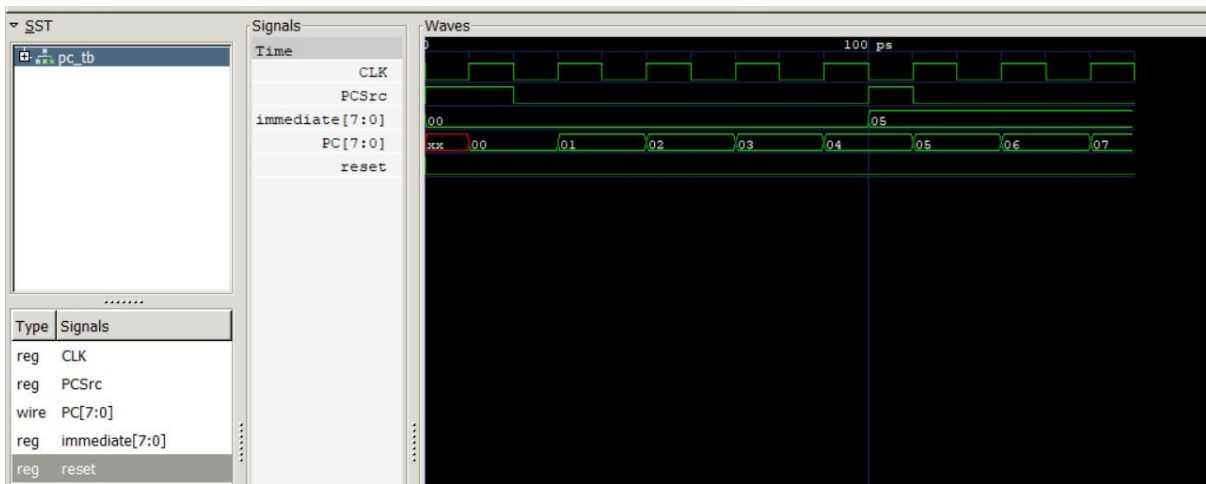
```
    $monitor("time = %3d, PCSrc = %b, immediate = %d, PC = %h", $time,  
    PCSrc, immediate, PC); end
```


endmodule

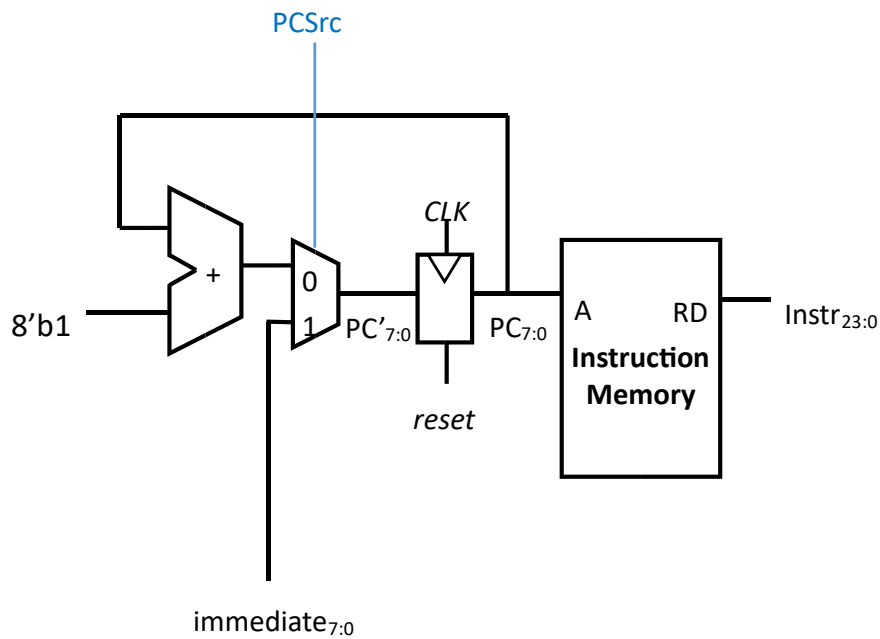
Test Result:

```
PS C:\Users\13452\Documents\Coursework files\ELEC0010> iverilog -g 2012 -o pc_tb.vvp pc_tb.sv
PS C:\Users\13452\Documents\Coursework files\ELEC0010> vvp pc_tb.vvp
VCD info: dumpfile pc_tb.vcd opened for output.
time = 0, PCSrc = 1, immediate = 0, PC = xx
time = 10, PCSrc = 1, immediate = 0, PC = 00
time = 20, PCSrc = 0, immediate = 0, PC = 00
time = 30, PCSrc = 0, immediate = 0, PC = 01
time = 50, PCSrc = 0, immediate = 0, PC = 02
time = 70, PCSrc = 0, immediate = 0, PC = 03
time = 90, PCSrc = 0, immediate = 0, PC = 04
time = 100, PCSrc = 1, immediate = 5, PC = 04
time = 110, PCSrc = 0, immediate = 5, PC = 05
time = 130, PCSrc = 0, immediate = 5, PC = 06
time = 150, PCSrc = 0, immediate = 5, PC = 07
pc_tb.sv:40: $finish called at 160 (1ps)
PS C:\Users\13452\Documents\Coursework files\ELEC0010> |
```

gtkwave:



6. Combining the ROM with program counter



Module name	instruction_memory_pc
Output ports	Instr[23:0]
Input ports	immediate[7:0], PCSrc, CLK, reset

Module Code:

```

`include "instruction_memory.sv"
`include "pc.sv"

module instruction_memory_pc (input logic [7:0] immediate,
input logic PCSrc, CLK, reset,          output
logic [23:0] Instr);

    logic [7:0] PC;

    pc pc_in (immediate,PCSrc, CLK, reset,PC);

    instruction_memory instruction_memory_in (PC,Instr);

endmodule

```

Test Bench Code:

```

`timescale 1ps/1ps

`include "instruction_memory.sv"

module instruction_memory_tb;
  logic [7:0] PC; logic [23:0]
  Instr;

  instruction_memory dut (PC,Instr);

  initial begin
    $dumpfile("instruction_memory_tb.vcd");
    $dumpvars(0,instruction_memory_tb);

    PC = 8'h00;
    #10 PC = 8'h01;
    #10 PC = 8'h02;
    #10 PC = 8'h03;
    #10 PC = 8'h04;
    #10 PC = 8'h05;
    #10 PC = 8'h06;
    #10 PC = 8'h07;

    $finish; end

  initial begin

    $monitor("time = %3d, PC = %d, Instr = %h", $time,PC,Instr); end

endmodule

```

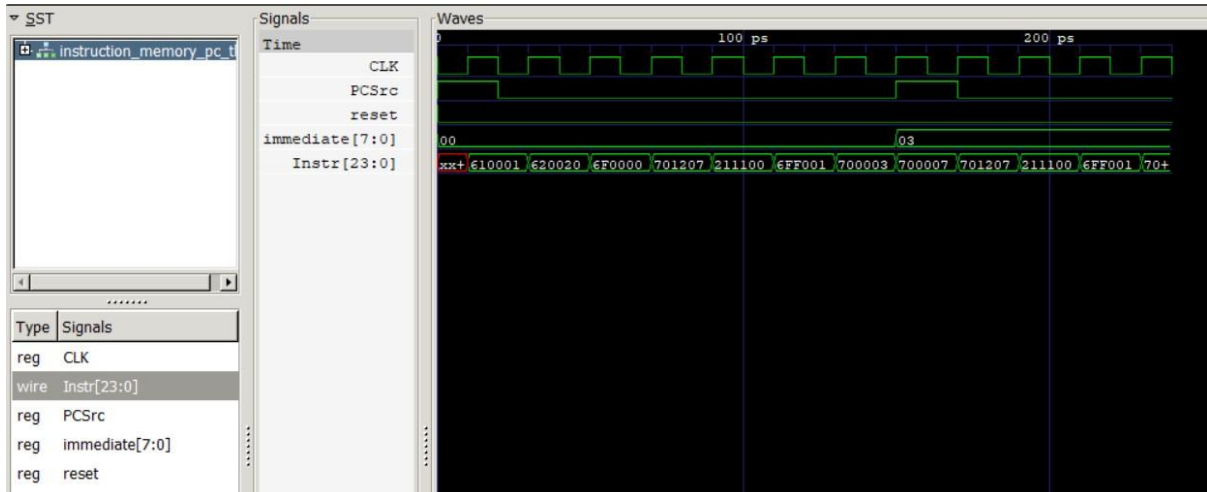
Test Result:

```

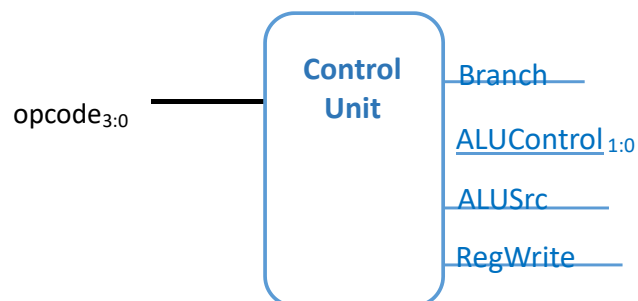
PS C:\Users\13452\Documents\Coursework files\ELEC0010> iverilog -g 2012 -o instruction_memory_pc_tb.vvp instruction_memory_pc_tb.sv
PS C:\Users\13452\Documents\Coursework files\ELEC0010> vvp instruction_memory_pc_tb.vvp
WARNING: ./instruction_memory.sv:7: $readmemh(program.txt): Not enough words in the file for the requested range [0:255].
VCD info: dumpfile instruction_memory_pc_tb.vcd opened for output.
time = 0, PCSrc = 1, immediate = 00, Instr = xxxxxx
time = 10, PCSrc = 1, immediate = 00, Instr = 610001
time = 20, PCSrc = 0, immediate = 00, Instr = 610001
time = 30, PCSrc = 0, immediate = 00, Instr = 620020
time = 50, PCSrc = 0, immediate = 00, Instr = 6f0000
time = 70, PCSrc = 0, immediate = 00, Instr = 701207
time = 90, PCSrc = 0, immediate = 00, Instr = 211100
time = 110, PCSrc = 0, immediate = 00, Instr = 6ff001
time = 130, PCSrc = 0, immediate = 00, Instr = 700003
time = 150, PCSrc = 1, immediate = 03, Instr = 700007
time = 170, PCSrc = 0, immediate = 03, Instr = 701207
time = 190, PCSrc = 0, immediate = 03, Instr = 211100
time = 210, PCSrc = 0, immediate = 03, Instr = 6ff001
time = 230, PCSrc = 0, immediate = 03, Instr = 700003
instruction_memory_pc_tb.sv:51: $finish called at 240 (1ps)
PS C:\Users\13452\Documents\Coursework files\ELEC0010>

```

gtkwave:



7. Control unit



Module name	control_unit
Output ports	Branch, ALUControl[1:0], ALUSrc, RegWrite
Input ports	opcode[3:0]

<i>Instruction</i>	<i>Op code</i>	<i>RegWrite</i>	<i>ALUSrc</i>	<i>ALUControl_{1:0}</i>	<i>Branch</i>
and	0000	1	0	00	0
or	0001	1	0	01	0
add	0010	1	0	10	0
sub	0011	1	0	11	0
andi	0100	1	1	00	0
ori	0101	1	1	01	0
addi	0110	1	1	10	0
beq	0111	0	0	11	1

Module Code:

```

module control_unit (input logic [3:0] opcode,
output logic [1:0] ALUControl,          output
logic Branch, ALUSrc, RegWrite);

    always_comb begin

        case (opcode)
            4'b0000: begin RegWrite = 1 ;ALUSrc = 0 ;ALUControl = 2'b00 ;Branch =
0 ; end //and
            4'b0001: begin RegWrite = 1 ;ALUSrc = 0 ;ALUControl = 2'b01 ;Branch =
0 ; end //or
            4'b0010: begin RegWrite = 1 ;ALUSrc = 0 ;ALUControl = 2'b10 ;Branch =
0 ; end //add
            4'b0011: begin RegWrite = 1 ;ALUSrc = 0 ;ALUControl = 2'b11 ;Branch =
0 ; end //sub
            4'b0100: begin RegWrite = 1 ;ALUSrc = 1 ;ALUControl = 2'b00 ;Branch =
0 ; end //andi
            4'b0101: begin RegWrite = 1 ;ALUSrc = 1 ;ALUControl = 2'b01 ;Branch =
0 ; end //ori
            4'b0110: begin RegWrite = 1 ;ALUSrc = 1 ;ALUControl = 2'b10 ;Branch =
0 ; end //addi
            4'b0111: begin RegWrite = 0 ;ALUSrc = 0 ;ALUControl = 2'b11 ;Branch =
1 ; end //beq
            default: begin RegWrite = 1'bx ;ALUSrc = 1'bx ;ALUControl =
2'bx ;Branch = 1'bx ;end
        endcase

```

end

endmodule

Test Bench Code:

`timescale 1ps/1ps

`include "control_unit.sv"

module control_unit_tb; **logic**

[3:0] opcode; **logic** [1:0]

ALUControl; **logic** Branch,

ALUSrc, RegWrite;

control_unit dut (opcode,ALUControl,Branch,ALUSrc,RegWrite);

initial begin

\$dumpfile("control_unit_tb.vcd");

\$dumpvars(0,control_unit_tb);

#10 opcode = 4'b0000;

#10 opcode = 4'b0001;

#10 opcode = 4'b0010;

#10 opcode = 4'b0011;

#10 opcode = 4'b0100;

#10 opcode = 4'b0101;

#10 opcode = 4'b0110;

#10 opcode = 4'b0111;

\$finish;

end

initial begin

```

$monitor("time = %3d, opcode = %b,ALUControl =%b ,Branch = %b,ALUSrc
= %b,RegWrite = %b",$time,opcode,ALUControl,Branch,ALUSrc,RegWrite); end

endmodule

```

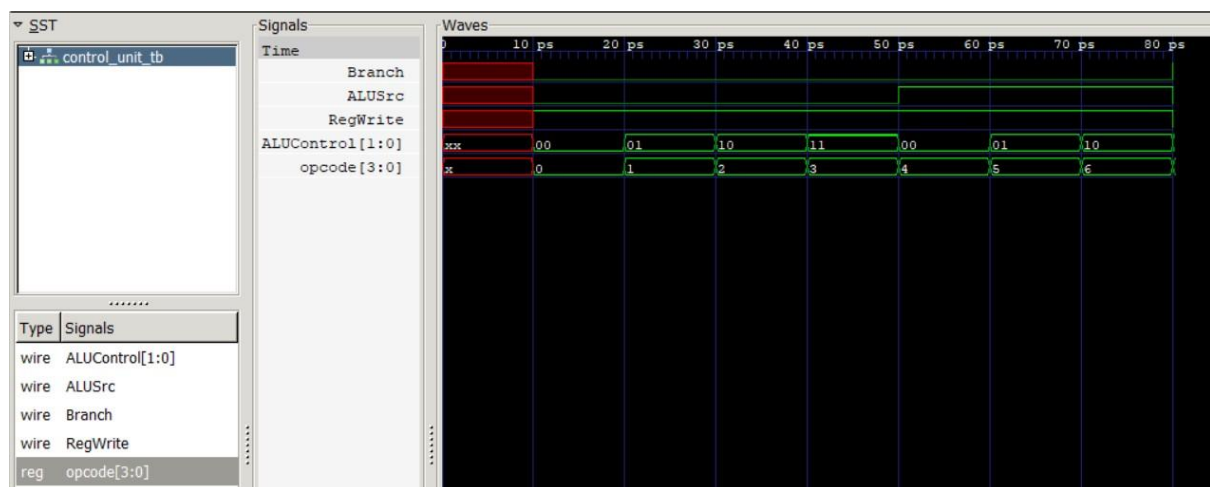
Test Result:

```

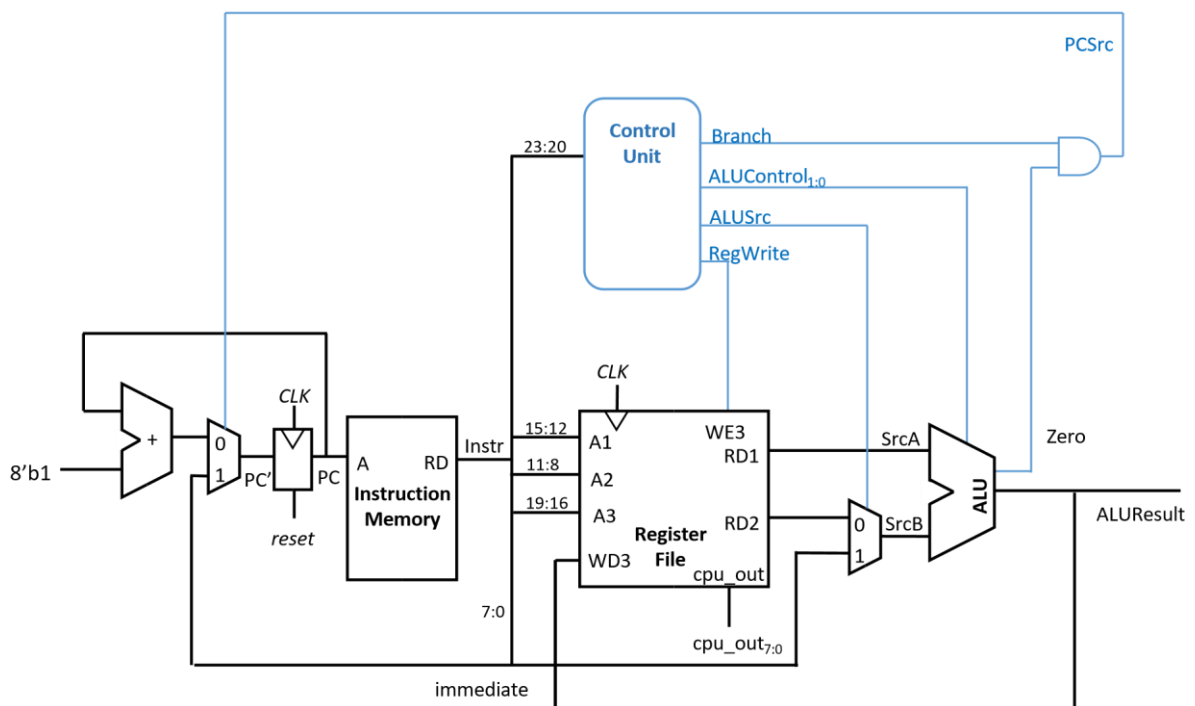
PS C:\Users\13452\Documents\Coursework files\ELEC0010> iverilog -g 2012 -o control_unit_tb.vvp control_unit_tb.sv
PS C:\Users\13452\Documents\Coursework files\ELEC0010> vvp control_unit_tb.vvp
VCD info: dumpfile control_unit_tb.vcd opened for output.
time = 0, opcode = xxxx,ALUControl =xx ,Branch = x,ALUSrc = x,RegWrite = x
time = 10, opcode = 0000,ALUControl =00 ,Branch = 0,ALUSrc = 0,RegWrite = 1
time = 20, opcode = 0001,ALUControl =01 ,Branch = 0,ALUSrc = 0,RegWrite = 1
time = 30, opcode = 0010,ALUControl =10 ,Branch = 0,ALUSrc = 0,RegWrite = 1
time = 40, opcode = 0011,ALUControl =11 ,Branch = 0,ALUSrc = 0,RegWrite = 1
time = 50, opcode = 0100,ALUControl =00 ,Branch = 0,ALUSrc = 1,RegWrite = 1
time = 60, opcode = 0101,ALUControl =01 ,Branch = 0,ALUSrc = 1,RegWrite = 1
time = 70, opcode = 0110,ALUControl =10 ,Branch = 0,ALUSrc = 1,RegWrite = 1
control_unit_tb.sv:24: $finish called at 80 (1ps)
time = 80, opcode = 0111,ALUControl =11 ,Branch = 1,ALUSrc = 0,RegWrite = 0
PS C:\Users\13452\Documents\Coursework files\ELEC0010>

```

gtkwave:



8. Complete CPU design



Module name	cpu
Output ports	ALUResult[7:0], cpu_out[7:0]
Input ports	CLK, reset

Module Code:

```

`include "reg_file_alu.sv"
`include "instruction_memory_pc.sv"
`include "control_unit.sv"

```



```

module cpu (input logic CLK, reset,
output logic [7:0] ALUResult, cpu_out);

    logic [3:0] RA1, RA2, WA, opcode;
logic [7:0] immediate;    logic [1:0]
ALUControl;

    logic write_enable, ALUSrc, Zero, PCSrc, Branch, RegWrite;
logic [23:0] Instr;

    assign immediate = Instr[7:0];
assign RA2 = Instr[11:8];    assign
RA1 = Instr[15:12];    assign WA =
Instr[19:16];    assign opcode =
Instr[23:20];

    assign write_enable = RegWrite;

    assign PCSrc = Branch & Zero;

    reg_file_alu reg_file_alu_in
(RA1,RA2,WA,immediate,ALUControl,write_enable, ALUSrc,
CLK,ALUResult,cpu_out,Zero);

    instruction_memory_pc instruction_memory_pc_in (immediate,PCSrc, CLK,
reset,Instr);

    control_unit control_unit_in (opcode,ALUControl,Branch, ALUSrc, RegWrite);

endmodule

```

Test Bench Code:

```

`timescale 1ps/1ps
`include "cpu.sv"

module cpu_tb; logic
CLK, reset;

```

```
logic [7:0] ALUResult, cpu_out;
```

```
cpu dut (CLK,reset,ALUResult,cpu_out);
```

```
initial    begin
```

```
CLK      =    0;
```

```
forever   begin
```

```
#10 CLK = ~CLK;
```

```
end end
```

```
initial begin
```

```
    $dumpfile("cpu_tb.vcd");
```

```
    $dumpvars(0,cpu_tb);
```

```
    reset = 1;
```

```
    #10
```

```
    #10 reset = 0;
```

```
    #510
```

```
    $finish; end
```

```
initial begin
```

```
    $monitor("time = %3d, ALUResult = %d, cpu_out = %d", $time, ALUResult,  
cpu_out); end
```

```
endmodule
```

Test Result:

```

PS C:\Users\13452\Documents\Coursework files\ELEC0010> vvp cpu_tb.vvp
WARNING: ./instruction_memory.sv:7: $readmemh(program.txt): Not enough words in the file for the requested range [0:255].
VCD info: dumpfile cpu_tb.vcd opened for output.
time = 0, ALUResult = x, cpu_out = x
time = 10, ALUResult = 1, cpu_out = x
time = 30, ALUResult = 32, cpu_out = x
time = 50, ALUResult = 0, cpu_out = x
time = 70, ALUResult = 225, cpu_out = x
time = 90, ALUResult = 2, cpu_out = 0
time = 110, ALUResult = 1, cpu_out = 0
time = 130, ALUResult = 0, cpu_out = 0
time = 150, ALUResult = 226, cpu_out = 1
time = 170, ALUResult = 4, cpu_out = 1
time = 190, ALUResult = 2, cpu_out = 1
time = 210, ALUResult = 0, cpu_out = 1
time = 230, ALUResult = 228, cpu_out = 2
time = 250, ALUResult = 8, cpu_out = 2
time = 270, ALUResult = 3, cpu_out = 2
time = 290, ALUResult = 0, cpu_out = 2
time = 310, ALUResult = 232, cpu_out = 3
time = 330, ALUResult = 16, cpu_out = 3
time = 350, ALUResult = 4, cpu_out = 3
time = 370, ALUResult = 0, cpu_out = 3
time = 390, ALUResult = 240, cpu_out = 4
time = 410, ALUResult = 32, cpu_out = 4
time = 430, ALUResult = 5, cpu_out = 4
time = 450, ALUResult = 0, cpu_out = 4
time = 470, ALUResult = 0, cpu_out = 5
cpu_tb.sv:27: $finish called at 530 (1ps)
PS C:\Users\13452\Documents\Coursework files\ELEC0010>

```

gtkwave:

